

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Disciplina: Teste de Software

Análise de Eficácia de Testes com Teste de Mutação

Aluna: **Gabriela Alvarenga Cardoso**

Matrícula: **844469**

Professor: Cleiton Tavares

Repositório: <https://github.com/gabialvarenga/operacoes-mutante>

Outubro de 2026

1. Análise Inicial

O projeto possui 50 funções em `src/operacoes.js` e 50 testes em `test/operacoes.test.js`, um para cada função. Todos os testes passam e a cobertura gerada pelo Jest é de 98,64% das linhas e 100% das funções.

```
Windows PowerShell — operacoes-mutante
PS C:\...\operacoes-mutante> npm test -- --coverage
-----
File          | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----
All files     | 85.41   | 58.82    | 100     | 98.64   |
operacoes.js | 85.41   | 58.82    | 100     | 98.64   | 112
-----
Test Suites: 1 passed, 1 total
Tests:       50 passed, 50 total
Snapshots:   0 total
Time:        1.579 s, estimated 2 s
Ran all test suites.
```

Figura 1 – Cobertura de código inicial.

Com o StrykerJS configurado para usar o Jest, a primeira execução gerou 213 mutantes. Foram detectados 159 (142 mortos e 17 por timeout), 42 sobreviveram e 12 não foram executados por nenhum teste, resultando em um **mutation score de 74,65%**.

159												42	12
File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total	
	Of total	Of covered											
All files	74.65	79.10	142	42	17	12	0	0	0	159	54	213	
JS operacoes.js	74.65	79.10	142	42	17	12	0	0	0	159	54	213	

Figura 2 – Resultado da primeira execução do Stryker.

Discrepância: a cobertura mede apenas se uma linha foi executada; ela não indica se existe alguma asserção capaz de perceber um resultado errado naquela linha. Na suíte original, quase todas as funções recebem uma única entrada "segura", como `isMaiorQue(10, 5)`, `isPar(100)` e `fatorial(4)`. Valores de fronteira, arrays vazios e respostas false nunca são verificados, e, entre as entradas inválidas, só a divisão por zero é testada. Assim, a linha é contada como coberta, mas um defeito nela passa despercebido, o que explica a diferença de cerca de 24 pontos entre os dois indicadores.

2. Análise de Mutantes Críticos

2.1 isMaiorQue: `a > b` substituído por `a >= b`

```
101 function areaCirculo(raio) { return Math.PI * Math.pow(raio, 2); }
102 function areaRetangulo(base, altura) { return base * altura; }
103 function perimetroRetangulo(base, altura) { return 2 * (base + altura); }
104 - function isMaiorQue(a, b) { return a > b; } ● ▼
+ function isMaiorQue(a, b) { return a >= b; }
105 function isMenorQue(a, b) { return a < b; } ● ●
106 function isEqual(a, b) { return a === b; } ●
107 function medianaArray(numeros) {
```

Figura 3 – Mutante de operador relacional sobrevivente na linha 104.

O que a mutação fez: trocou o operador relacional, simulando um erro na fronteira da comparação. **Por que o teste não o matou:** o único teste chama `isMaiorQue(10, 5)`, e para esse par tanto `>` quanto `>=` retornam `true`. As duas versões só diferem quando `a` e `b` são iguais, entrada que não aparecia na suíte. Pelo mesmo motivo sobreviveram a troca análoga em `isMenorQue` e as versões que substituem o corpo por `return true`, já que nenhum teste esperava `false`.

2.2 medianaArray: chamada a `sort` removida

```
107 function medianaArray(numeros) {
108   if (numeros.length === 0) throw new Error('Array vazio He possui mediana. '); ● ●
109 - const sorted = [...numeros].sort((a, b) => a - b); ▼ ● ●
+ const sorted = [...numeros];
110 const mid = Math.floor(sorted.length / 2);
111 if (sorted.length % 2 === 0) { ● ● ●
112   return (sorted[mid - 1] + sorted[mid]) / 2; ● ● ●
```

Figura 4 – Mutante de remoção de método sobrevivente na linha 109.

O que a mutação fez: eliminou a ordenação, de modo que a função passa a escolher o elemento central na ordem original do array. **Por que o teste não o matou:** a entrada `[1, 2, 3, 4, 5]` já chega em ordem crescente, então ordenar ou não

leva ao mesmo resultado. As mutações no comparador ($a + b$ e $() \Rightarrow undefined$) sobreviveram pela mesma razão, e o ramo de arrays com tamanho par nunca foi executado, deixando sem cobertura os quatro mutantes gerados nele.

2.3 fatorial: $n < 0$ substituído por $n \leq 0$

```

16 function restoDivisao(dividendo, divisor) { return dividendo % divisor; }
17 function fatorial(n) {
18 - if (n < 0) throw new Error('Fatorial não é definido para números negativos.');
```

```

+ if (n <= 0) throw new Error('Fatorial não é definido para números negativos.');
```

```

19   if (n === 0 || n === 1) return 1;
20   let resultado = 1;
21   for (let i = 2; i <= n; i++) { resultado *= i; }
```

Figura 5 – Mutante de operador relacional sobrevivente na linha 18.

O que a mutação fez: fez a função rejeitar o zero como se fosse negativo, quebrando a definição $0! = 1$. **Por que o teste não o matou:** a suíte só calcula fatorial(4). Sem testes para zero ou para números negativos, a validação de entrada pode ser alterada, ou até desativada com `if (false)`, sem que nenhum teste falhe.

3. Solução Implementada

Os 50 testes originais foram mantidos e foram adicionados 34 testes em um novo bloco describe, totalizando 84. Para os mutantes analisados:

```

test('isMaiorQue deve retornar false para valores iguais', () => { expect(isMaiorQue(5, 5)).toBe(false); });
test('isMaiorQue deve retornar false quando a é menor que b', () => { expect(isMaiorQue(5, 10)).toBe(false); });

test('medianaArray deve ordenar o array antes de calcular (tamanho ímpar)', () => { expect(medianaArray([5, 1, 3])).toBe(3); });
test('medianaArray deve tirar a média dos dois elementos centrais (tamanho par)', () => { expect(medianaArray([4, 1, 3, 2])).toBe(2.5); });

test('fatorial de zero deve ser 1 (limite da validação)', () => { expect(fatorial(0)).toBe(1); });
test('fatorial deve lançar erro para número negativo', () => { expect(() => fatorial(-1)).toThrow('Fatorial não é definido para números negativos.');
```

- **isMaiorQue(5, 5):** o original devolve false e o mutante $\geq$ devolve true. O caso (5, 10) mata a versão `return true`.
- **medianaArray([5, 1, 3]):** o resultado correto (3) só é obtido se o array for ordenado; sem a ordenação, o elemento central seria 1. O array [4, 1, 3, 2] executa o ramo par, cujo resultado (2,5) muda com qualquer alteração em `mid - 1`, no + ou no / 2.
- **fatorial(0) e fatorial(-1):** o zero fica exatamente na fronteira entre $<$ e $\leq$, e o caso negativo garante que a validação existe e lança a mensagem correta.

Os demais testes seguem os mesmos critérios: respostas false de `isPar`, `isImpar`, `isDivisivel`, `isMenorQue` e `isEqual`; `isPrimo` com 0, 1, 2 e 9; arrays vazios em `mediaArray`, `maximoArray`, `minimoArray` e `produtoArray`; mensagens exatas em todos os novos `toThrow`; e conversões de temperatura com 100 °C e 212 °F, valores em que o fator 9/5 influencia o resultado. Em `clamp`, os testes usam valores iguais aos limites, com 0 e -0 (iguais para $<$, mas diferentes para o `toBe`), para verificar se a função devolve o próprio valor e não o limite. Em `produtoArray`, um array vazio com `reduce` sobrescrito garante que o retorno 1 vem da verificação de array vazio.

4. Resultados Finais

209												
File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
All files	98.12	98.12	206	4	3	0	0	0	0	209	4	213
JS operacoes.js	98.12	98.12	206	4	3	0	0	0	0	209	4	213

Figura 6 – Resultado da execução final do Stryker.

Métrica	Inicial	Final
Quantidade de testes	50	84
Cobertura de linhas / branches	98,64% / 58,82%	100% / 100%
Mutantes detectados (mortos + timeout)	159	209
Sobreviventes / sem cobertura / ignorados	42 / 12 / 0	4 / 0 / 0

Mutation score	74,65%	98,12%
----------------	--------	--------

O mutation score passou de 74,65% para **98,12%**, acima da meta de 98%, sem ignorar nenhum mutante e sem alterar o código de `src/operacoes.js`. Os 4 mutantes restantes estão na linha 19 (`if (n === 0 || n === 1) return 1`) e são equivalentes: para 0 e 1 o laço seguinte não executa e a função já retorna 1, então remover ou alterar essa condição não muda nenhum resultado.

5. Conclusão

O teste de mutação mostrou que uma suíte com quase toda a cobertura possível ainda deixava passar um em cada quatro defeitos simulados. Isso acontece porque a cobertura avalia o que foi executado, enquanto o teste de mutação avalia o que foi verificado: cada mutante sobrevivente representa um bug que os testes não perceberiam. Além de oferecer uma métrica mais fiel da qualidade da suíte, o relatório do Stryker orienta a melhoria, apontando com precisão quais fronteiras, ramos e respostas ainda não são testados. Apesar do custo maior de execução, é uma ferramenta valiosa para garantir que os testes realmente protegem o código.